Reflexions- und Entwurfsdokument – IU Studien-Dashboard

Modul: Objektorientierte und funktionale Programmierung mit Python (DLBDSOOFPP01_D)

Phase: 2 – Erarbeitungs-/Reflexionsphase

Name: Sascha Schumbera

Matrikelnummer: IU14153477

Datum: April 2026

1. Umsetzung objektorientierter Konzepte in Python

Das Klassendiagramm aus Phase 1 war mein Ausgangspunkt. Für jedes OOP-Konzept habe ich ein kleines Testskript geschrieben (phase2_test_*.py), um zu prüfen, ob sich das Diagramm in Python so umsetzen lässt wie geplant – oder ob Anpassungen nötig sind. Die zentrale Frage dieser Phase war also: Was lässt sich direkt übernehmen, und wo muss ich etwas anpassen?

1.1 Enumeration – ModulStatus

Das Attribut status in Modul habe ich auf Empfehlung von Aiko Walte (Tutor) als Enumeration modelliert, damit man nicht aus Versehen falsche Begriffe oder Tippfehler beim Status eingeben kann. In Python gibt es dafür enum.Enum aus der Standardbibliothek (phase2_test_enum.py):

```
from enum import Enum
class ModulStatus(Enum):
    GEPLANT          = "Geplant"
    IN_BEARBEITUNG   = "In Bearbeitung"
    BESTANDEN        = "Bestanden"
    NICHT_BESTANDEN  = "Nicht bestanden"
```

Ergebnis: Ungültige Zuweisungen werfen automatisch einen ValueError – das muss nicht manuell abgefangen werden. Alle Zustände lassen sich über eine Schleife auflisten, was für spätere Dropdown-Menüs nützlich ist. Das Klassendiagramm konnte hier ohne Änderungen übernommen werden.

1.2 Kapselung – @property statt Getter/Setter

Im Diagramm sind alle Attribute als privat markiert. Da Python keinen echten Schreibschutz für Variablen hat, habe ich einen Unterstrich vor den Namen gesetzt (z. B. _note) – das ist eine Konvention, die signalisiert: nicht direkt von außen ändern. Für den Zugriff habe ich @property verwendet statt klassische get/set-Methoden (phase2_test_property.py):

```
class Pruefungsleistung:
    @property
    def note(self) -> float:
        return self._note
    @note.setter
    def note(self, wert: float) -> None:
        if not 1.0 <= wert <= 5.0:
            raise ValueError(f"Note {wert} ungültig (1.0-5.0)")
        self._note = wert
```

Ergebnis: @property ist ein Implementierungsdetail und erscheint nicht im UML. Die Validierung sitzt direkt im Setter – ich muss also nicht an jeder Stelle im Code extra prüfen, ob eine Note gültig ist.

Information Expert Prinzip: Aiko Walte (Tutor) hat darauf hingewiesen, dass get_note() zur Klasse Pruefungsleistung gehört, weil dort die Note gespeichert ist. Die Note im Modul abzufragen wäre falsch – das Modul hat sie gar nicht. In der Umsetzung ist note deshalb als @property in Pruefungsleistung untergebracht.

1.3 Vererbung – Abschlussarbeit erbt von Modul

Vererbung wird in Python durch Angabe der Elternklasse in Klammern umgesetzt. Mit super().__init__() werden die gemeinsamen Attribute an Modul weitergegeben – so muss der Konstruktor von Modul nicht doppelt geschrieben werden (phase2_test_vererbung.py):

```
class Abschlussarbeit(Modul):
    def __init__(self, thema: str, betreuer: str) -> None:
        super().__init__("Bachelorarbeit", "DLBABSCHLUSSB01", ects=12)
        self._thema = thema
        self._betreuer = betreuer
```

Ergebnis: Im Test erzeuge ich ein Abschlussarbeit-Objekt (ba) und prüfe mit isinstance(ba, Modul), ob es überall dort eingesetzt werden kann, wo ein normales Modul erwartet wird – das funktioniert.

Design-Entscheidung: Die Werte ects=12 und name="Bachelorarbeit" sind im Konstruktor fest eingetragen – eine bewusste Vereinfachung für den Prototypen. Für spätere Erweiterungen (z. B. Masterstudiengang mit 30 ECTS) müssten diese Werte als Parameter übergeben werden.

Hinweis von Aiko Walte (Tutor): Die Abschlussarbeit darf laut Kardinalität aus Phase 1 (0..1) nur einmal pro Studiengang vorkommen. Python erzwingt das nicht von sich aus – die Prüfung gehört in die Klasse Studiengang, nicht in die Oberfläche. Es wäre unsauber, das erst in der Benutzeroberfläche zu prüfen, weil die Regel für das gesamte System gelten muss – egal ob die Daten über die App oder später z. B. per Datei-Import reinkommen. modul_hinzufuegen() prüft deshalb, ob bereits eine Abschlussarbeit vorhanden ist, und wirft bei Verstoß eine ValueError-Exception (phase2_test_komposition.py):

```
def modul_hinzufuegen(self, modul, semester):
    if isinstance(modul, Abschlussarbeit):
        alle = [m for s in self._semester for m in s.module]
        if any(isinstance(m, Abschlussarbeit) for m in alle):
            raise ValueError("Nur eine Abschlussarbeit pro Studiengang.")
    semester.modul_hinzufuegen(modul)
```

1.4 Komposition und Aggregation

In Python gibt es keinen speziellen Befehl für Komposition oder Aggregation – ich löse beides über Listen und Objektreferenzen (phase2_test_komposition.py). Im Code von 1.3 habe ich dafür auch eine List Comprehension eingesetzt, die alle Module über alle Semester abfragt – ein funktionales Konzept, das Python gut unterstützt. Der Unterschied zwischen Komposition und Aggregation liegt im Entwurf:

Komposition: Studiengang → Semester → Modul. Der Studiengang erzeugt das Semester selbst über erstelle_semester(), das Semester verwaltet seine Module intern. Wird der Studiengang gelöscht, verschwinden auch alle Semester und deren Module – der Lebenszyklus liegt vollständig beim jeweils übergeordneten Objekt. Das entspricht der Komposition, die im UML mit der ausgefüllten Raute (*--) dargestellt ist. Python hat dafür kein

eigenes Sprachkonstrukt, der Effekt wird durch die interne Verwaltung der Objektlisten erreicht.

```
def erstelle_semester(self, nummer, startdatum, enddatum):
    sem = Semester(nummer, startdatum, enddatum)
    self._semester.append(sem)
    return sem  # nur zur Nutzung im selben Kontext
```

Aggregation: Modul → Prüfungsleistung. Eine Pruefungsleistung hängt nicht zwingend am Modul – das Modul kennt sie, ist aber nicht für sie zuständig. Anders als bei der Komposition verschwindet die Pruefungsleistung nicht automatisch, wenn das Modul wegfällt.

Fazit: Das Klassendiagramm aus Phase 1 konnte ich im Wesentlichen direkt übernehmen. Anpassungen gab es nur auf technischer Ebene: Getter und Setter habe ich durch @property ersetzt. Die Kardinalitätsprüfung für die Abschlussarbeit (0..1) habe ich explizit in modul_hinzufuegen() eingebaut, da Python das nicht automatisch erzwingt. An der fachlichen Struktur der Klassen selbst hat sich nichts geändert.

2. Gesamtarchitektur

Die fünf Entity-Klassen aus Phase 1 bleiben der fachliche Kern. Für den Prototypen kommen Persistenz, Domänenlogik (KPI), eine Vermittlungsschicht und die Oberfläche hinzu. Gegenüber der ersten Entwurfsversion gliedert sich die Architektur jetzt in fünf Pakete: Entity-Klassen, Domain-Services (inkl. KPI-Wertobjekte), Infrastruktur, Anwendungsschicht (Controller) und UI. Damit werden die Hinweise aus dem Tutor-Feedback zur klaren Verantwortungstrennung und zur Austauschbarkeit der Persistenz berücksichtigt.

2.1 Klassen und Aufgaben

StudiengangRepository (Schnittstelle): Abstrakte Schnittstelle mit laden() und speichern(studiengang). Sie beschreibt nur den fachlichen Vertrag („einen Studiengang lesen/schreiben“) und ist nicht an JSON gebunden. Die konkrete Klasse JSONRepository implementiert diese Schnittstelle. Der Controller hängt von der Schnittstelle ab, nicht von JSONRepository direkt (Dependency-Inversion) – die UI kennt weder JSONRepository noch StudiengangRepository. Ein Wechsel der Speicherung erfordert nur eine neue Implementierung, ohne Änderungen in UI oder Controller.

JSONRepository: Konkrete Persistenz in einer JSON-Datei; implementiert StudiengangRepository.

KPICalculator: Berechnet die vier Dashboard-KPIs aus dem Studiengang-Objekt. Statt generischer dictionaries liefern die Methoden jetzt klar benannte Wertobjekte (z. B. RegelstudienzeitKpi) mit Feldern wie Titel, Hauptwert, Zielhinweis und ziel_erreicht. Das erhöht Lesbarkeit und reduziert Tippfehler beim Zugriff auf Einzelwerte in der UI.

Python-Reflexion KPI-Typen: Die KPI-Wertobjekte sind in Python als @dataclass realisiert. Gegenüber einem gewöhnlichen dict hat das drei konkrete Vorteile: Erstens sind die Felder statisch benannt – der IDE-Autovervollständigungs-Support greift sofort, und ein Tippfehler wie kpi['zeil_erreicht'] führt zur Laufzeit zu einem AttributeError statt still einen falschen Wert zu liefern. Zweitens lassen sich Typ-Annotationen direkt am Feld notieren (titel: str, ziel_erreicht: bool), sodass ein Typprüfer wie mypy Fehler vor der Ausführung meldet. Drittens dokumentiert die Klassendefinition selbst, welche Felder ein KPI-Objekt hat – ein dict gibt dazu keinen Hinweis ohne Kommentar oder Konvention. Im Vergleich zu einem

NamedTuple bietet @dataclass den Vorteil, dass Felder später um Methoden (z. B. formatierte Ausgabe) erweiterbar sind, ohne die Schnittstelle zu ändern.

```
from dataclasses import dataclass
@dataclass
class RegelstudienzeitKpi:
    titel: str
    hauptwert: str
    zielhinweis: str
    ziel_erreicht: bool
# Zugriff: kpi.ziel_erreicht  (statt kpi['ziel_erreicht'])
```

DashboardController: Kapselt alle Anwendungsfälle: initialer Ladevorgang, Speichern nach Änderungen, Modul hinzufügen (inkl. automatischer Semester-Anlage), Prüfungsleistung eintragen sowie die vier KPI-Abfragen (Delegation an KPICalculator). Die UI spricht ausschließlich mit dem Controller und kennt kein konkretes JSONRepository – das entlastet DashboardApp und verbessert die Testbarkeit (Controller lässt sich mit Fake-Repository prüfen). Während der Umsetzung des UML-Diagramms habe ich festgestellt, dass noch weitere Methoden hinzugefügt werden sollten, um die Nutzerfahrung zu optimieren und mögliche Fehler selbstständig bereinigen zu können. (modul_entfernen, semester_anlegen, semester_entfernen, daten_exportieren (siehe Dashboard-Skizze aus Phase1)

DashboardApp: Zentrale tkinter-Klasse: Aufbau von KPI-Leiste, Modultabelle und Navigation. Sie hält eine Referenz auf den DashboardController und delegiert Daten- und Speicheroperationen dorthin. Dialoge (ModulDialog, EingabeDialog) bleiben ausgelagert.

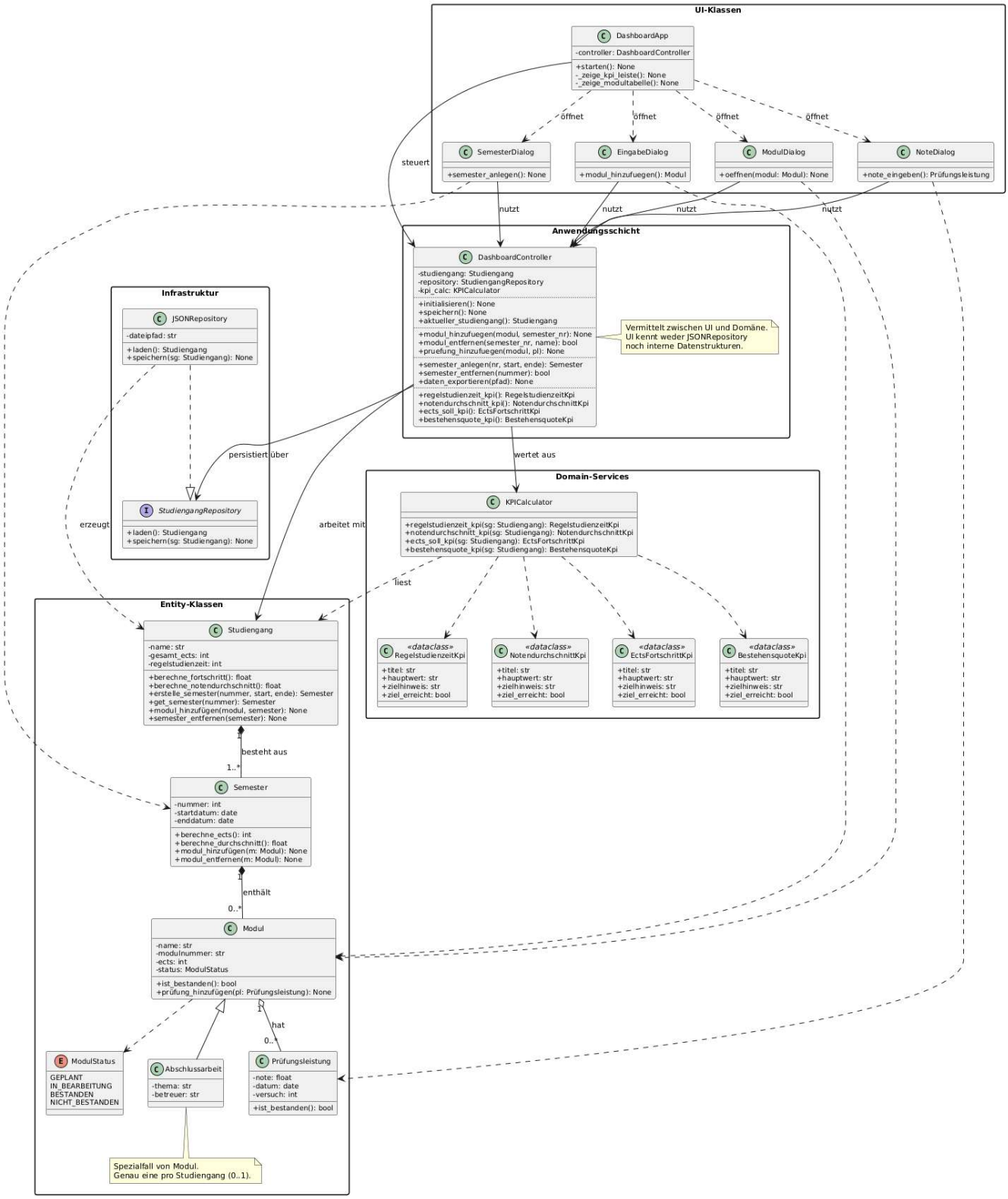
ModulDialog: Detailfenster zu einem Modul inkl. Prüfungsversuche.

EingabeDialog: Erfasst neue Module (Name, Modulnummer, ECTS, Zielsemester). Die UI übergibt das erzeugte Modul-Objekt an den Controller, der Domänenregeln (z. B. Abschlussarbeit 0..1) und Persistenz orchestriert.

NoteDialog: Erfasst neue Prüfungsleistungen (Modulauswahl, Note, Datum, Versuchsnummer). Die Trennung von EingabeDialog und NoteDialog folgt dem Single-Responsibility-Prinzip: jede Dialog-Klasse hat genau eine fachliche Aufgabe. Die UI reicht die Prüfungsleistung an den Controller weiter, der Status-Aktualisierung und Speicherung übernimmt.

SemesterDialog: Die Phase-1-Skizze sieht den Button „+ Semester anlegen“ vor. In Version 2 des Diagramms fehlte dafür eine eigene Dialog-Klasse – die Funktionalität war nur implizit im Controller geplant. In der Implementierung wurde SemesterDialog als eigenständige tkinter-Klasse realisiert, analog zu EingabeDialog und NoteDialog. Sie folgt demselben Single-Responsibility-Prinzip: jede Dialog-Klasse hat genau eine fachliche Aufgabe (hier: Eingabe von Semesternummer, Start- und Enddatum). DashboardApp öffnet den Dialog und übergibt das Ergebnis an den Controller – die Schichtentrennung bleibt erhalten.

2.2 UML-Klassendiagramm Gesamtarchitektur

Das Diagramm uml_gesamtarchitektur.puml wurde mit PlantUML erstellt. Es zeigt die Schnittstelle StudiengangRepository, die Implementierung JSONRepository, die KPI-Wertklassen, den DashboardController sowie die UI-Klassen. Abhängigkeiten verlaufen weiterhin von außen nach innen; Entity-Klassen kennen weder Repository noch Oberfläche. Zur Unterstützung der neu-implementierten Lösch-Flows wurden zwei Methoden ergänzt (**Semester:** modul_entfernen und **Studiengang:** semester_entfernen).

Hinweis zur Benennung: Das UML nutzt deutsche Bezeichner mit Umlauten; der Python-Code verwendet ASCII-Approximationen (z. B. Pruefungsleistung).